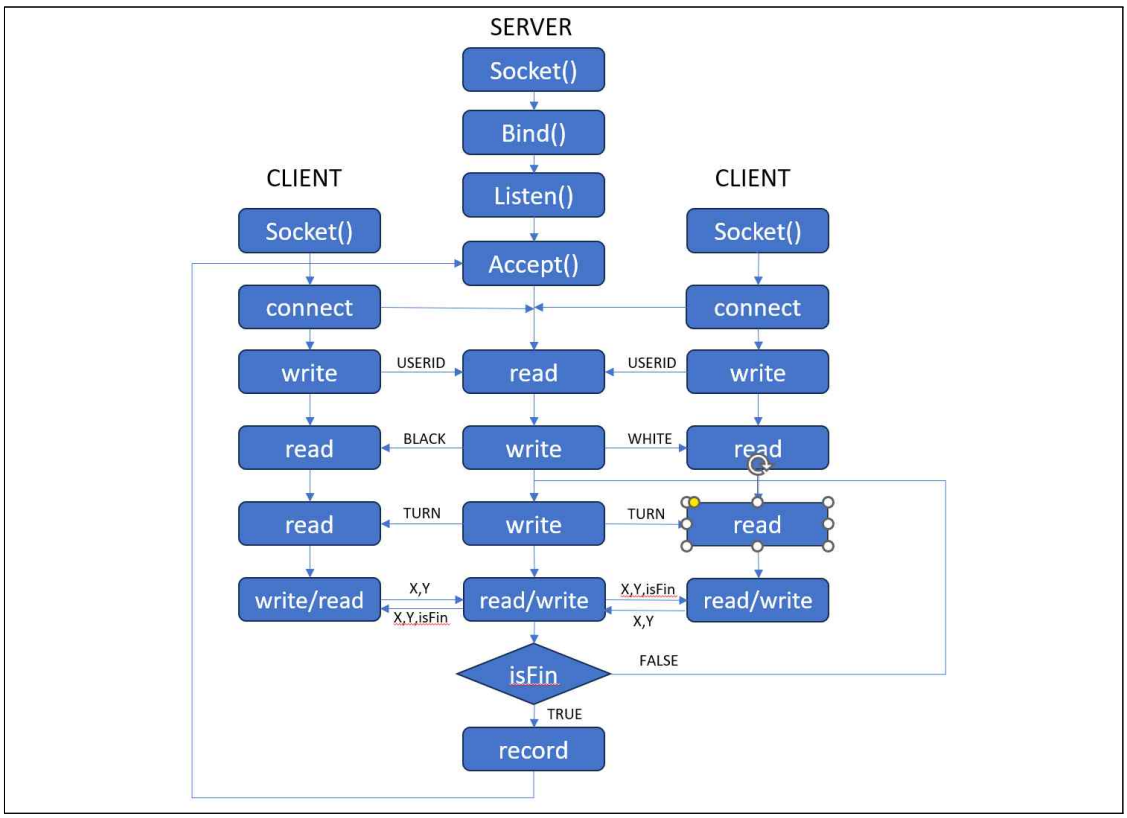
시스템 프로그래밍
TERM PROJECT REPORT

2022112332 강진우

세부 구조

●순서도



●설명

1. 서버에 두 클라이언트가 연결됨
2. 클라이언트는 서버에게 자신들의 UID를 전송하고 게임 화면을 띄움
3. 서버는 UID를 받아 배열에 저장하고 먼저 연결된 클라이언트에 1번(BLACK) 다른 쪽에 2번(WHITE)을 할당
4. 서버는 두 클라이언트에게 누구의 차례인지와 게임이 진행 중인지 전달
5. 클라이언트는 자신의 차례라면, 사용자로부터 좌표를 입력받아 화면에 돌을 표시한 후 서버에 좌표를 보냄
6. 클라이언트는 좌표를 받아 오목이 완성되었는지 판단 후 다른 클라이언트에 [게임이 진행 중인지, 누구의 차례인지, x좌표, y좌표]를 보냄
7. 자신의 차례가 아닌 클라이언트는 서버로부터 좌표를 받고 화면에 상대가 놓은 돌의 위치를 표시

8. 만약 오목이 완성되었으면, 두 클라이언트에 누가 이겼는지 전달
9. 클라이언트에서 결과에 따라 이겼는지 졌는지 표시 후 종료
10. 서버는 얻은 UID를 바탕으로 record.txt에 승리한 UID가 몇 번 승리했는지 기록

함수 및 변수설명

●Omok_Server.c

-signal(SIGINT,SIG_IGN): 서버이기 때문에 SIGINT로 인해 중단되지 않도록 시그널 무시
-board: 오목 배열

```
int checkFinish(int x, int y) {//0: 1: Black win, 2: White win
	int cur[2];
	int dir[8][2] = { {0,1},{1,1},{1,0},{1,-1},{0,-1},{-1,-1},{-1,0},{-1,1} };
	int updown = 0, rightleft = 0, crossL = 0, crossR = 0;
	for (int d = 0; d < 8; d++) {
		cur[0] = x; cur[1] = y;
		while (board[cur[1]][cur[0]] == board[y][x]) {
			cur[0] += dir[d][0]; cur[1] += dir[d][1];
			if (d == 0 || d == 4) {
				updown += 1;
			}
			else if (d == 1 || d == 5) {
				crossR += 1;
			}
			else if (d == 2 || d == 6) {
				rightleft += 1;
			}
			else if (d == 3 || d == 7) {
				crossL += 1;
			}
		}
	}
	if (updown > 5 || crossR > 5 || rightleft > 5 || crossL > 5)
		return board[y][x];
	return 0;
}
```

-checkFinish(int y, int x): 직전에 놓은 돌의 좌표를 기준으로 상하좌우, 대각선 방향으로 같은 돌이 5개 이상 연속으로 있는지 검사하고, 없으면 0, 있으면 1(BLACK) 또는 2(WHITE) 반환

```
int drawStone(int y, int x, int stone) { // stone이 1이면 흑, 2면 백
	if (board[y][x] || x < 0 || x >= MAX_N || y < 0 || y >= MAX_N)
		return 0;
	if (stone == BLACK) {
		board[y][x] = BLACK;
	}
	else if (stone == WHITE) {
		board[y][x] = WHITE;
	}
	else
		return 0;
	return 1;
}
```

-drawStone(int y, int x, int stone): 좌표와 돌의 색을 받아 배열에 표시

```
int checklist(LINK head, char* uid) {
	LINK nextnode = head;
	while (nextnode) {
		if (!strcmp(nextnode->userid, uid)) {
			nextnode->win += 1;
			return 1;
		}
		nextnode = nextnode->next;
	}
	return 0;

}
```

-checklist(LINK head, char* uid): 연결리스트를 받아 해당 리스트에 입력받은 uid와 같은 uid가 있는지 검사하고 있으면 해당 노드의 승수를 1 증가시키고 1을 반환, 없으면 0을 반환

```
void record(char* uid) {
	LINK head = NULL, cur;
	char temp[20];
	int win;
	char userid[10];
	int fd = open("record.txt", O_RDONLY, S_IRUSR | S_IWUSR);
	FILE* fp = fdopen(fd, "r");
	if (fp != NULL) {
		while (!feof(fp)) {
			initializeBuff(userid, 10);
			fscanf(fp, "%s%d", userid, &win);
			if (userid[0] != '\0') {
				cur = createNode(userid, win);
				head = appendNode(head, cur);
				printf("%s%d\n", userid, win);
			}

		}

		fclose(fp);
```

```
        }
        close(fd);
        if (!checklist(head, uid)) {
                cur = createNode(uid, 1);
                head = appendNode(head, cur);
        }

        fd = open("record.txt", O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR);
        fp = fdopen(fd, "w");
        if (fp == NULL)
                oops("fpwrite");
        cur = head;
        while (cur) {
                fprintf(fp, "%s %d\n", cur->userid, cur->win);
                cur = cur->next;
        }

        freeNode(head);
        fclose(fp);
        close(fd);
}
```

-record(char* userid): record.txt 파일을 열어(없으면 생성) uid와 승수를 구조체의 형태로 저장하여 연결리스트로 만듦. checklist()의 반환값이 0이면 새로운 노드를 생성하여 연결리스트에 append. record.txt 파일에 연결리스트에 있는 데이터를 출력하고 연결리스트들을 free 해줌.

●Omok_client.c

-curses를 이용해 새로운 창에서 게임이 진행됨

```
void inputCode(int* y, int* x) {
        gotoInput();
        addstr("Input Coordinate (A19): ");
        char s[10];
        getnstr(s, 4);
        if (!strcmp(s, "exit")) {
                endwin();
                exit(1);
        }

        *x = s[0] - 'A';
        if (strlen(s) > 2)
                return;
        else if (strlen(s) == 2) {
                *y = s[1] - 'a';
        }
        else
                *y = *x = -1;
        gotoInput(); addstr("Input Coordinate (Gm): \t\t\t");
        return;
}
```

-inputCode(int *y, int*x): getnstr()을 통해 사용자로부터 알파벳 대문자와 소문자로 이루어

진 문자열을 입력받음. 각 알파벳을 숫자로 변형해서 각각 y와 x에 저장

```
int drawStone(int y, int x, int stone) { // stone이 1이면 흑, 2면 백
	if (board[y][x] || x < 0 || x >= MAX_N || y < 0 || y >= MAX_N)
		return 0;
	gotoxy(x, y);
	if (stone == BLACK) {
		DRAW_BLACK;
		board[y][x] = BLACK;
	}
	else if (stone == WHITE) {
		DRAW_WHITE;
		board[y][x] = WHITE;
	}
	else
		return 0;
	refresh();
	return 1;
}
```

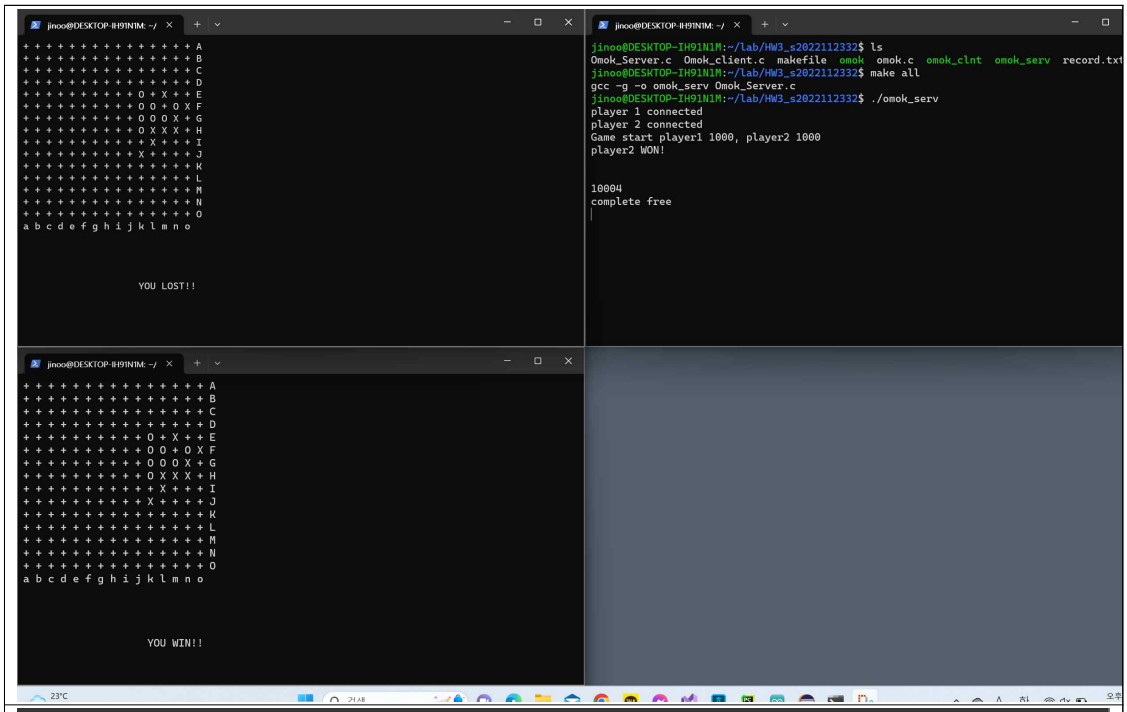
-drawStone(int y, int x, int stone): 입력받은 좌표대로 board 배열에 돌을 표시하고, 화면에도 표시. 만약 받은 좌표가 틀린 좌표라면 0을 반환. 정상적인 좌표면 1을 반환

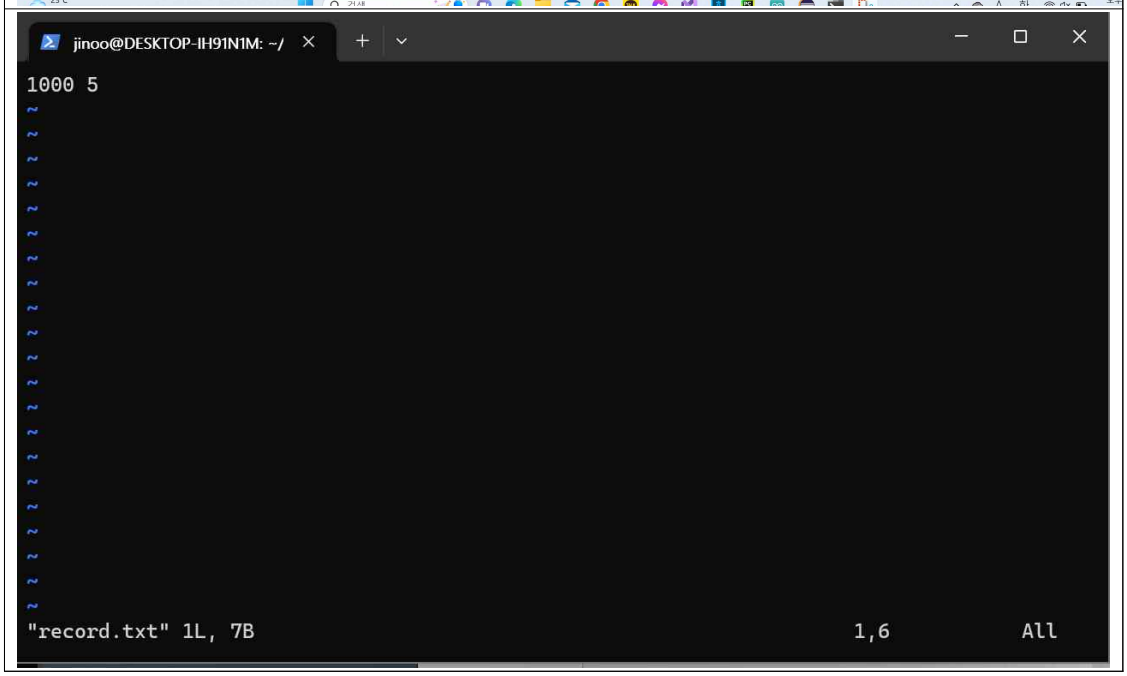
```
void drawInit() {
	gotoxy(0, 0);
	for (int i = 0; i < MAX_N; i++) {
		for (int j = 0; j < MAX_N; j++) {
			gotoxy(i, j);
			addstr("+ ");
			if (j == MAX_N - 1) {
				gotoxy(i, j + 1);
				addch(i + 'A');
			}
		}

	}
	for (int i = 0; i < MAX_N; i++) {
		gotoxy(MAX_N, i);
		addch(i + 'a');
	}
	gotoInput();
	addstr("fin.");
}
```

-drawInit(): 처음 바둑판을 화면에 표시

결과 화면





사용법

1. omok_server 실행
2. ./omok_client ServerName(IP 주소를 얻으려는 서버 이름 또는 서버에 대한 도메인명) 실행 x2
3. 자신이 돌을 놓고자 하는 위치를 대문자와 소문자의 조합(ex: Gm)으로 입력
4. 게임 진행

참고자료

-[c언어 오목] 세상 간단한 오목 만들기 ,tistory, 2020. 5. 10,
<https://hyomyo.tistory.com/61>
-[Network][TCP/IP] 소켓 구현(1)_ 리눅스 서버 구현, tistory, 2020. 10. 24,
<https://aronglife.tistory.com/entry/NetworkTCPIP-%EC%86%8C%EC%BC%93-%ED%94%84%EB%A1%9C%EA%B7%B8%EB%9E%98%EB%B0%8D2-%EC%84%9C%EB%B2%84-%EA%B5%AC%ED%98%84>
-[리눅스 프로그래밍] (IPC) Socket 통신 다루기, velog.io, 2022. 7. 6,
<https://velog.io/@palinyee12/리눅스-프로그래밍-IPC-Socket-통신-다루기>